Technical Documentation

PKCS#7 Signature Verification (signature_verify.c)

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

BMW Car IT, Ulm

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

File Location

`linux/drivers/md/dm-verity-autoboot/signature_verify.c`

1. Introduction and Architectural Role

The signature_verify.c module serves as the **cryptographic root of trust** for the dm-verity-autoboot system. It is the final and most critical security gate, ensuring that the dm-verity metadata used to establish the verified root filesystem has not been modified and was genuinely signed by a trusted authority. This module heavily relies on the Linux kernel's built-in cryptographic and signing infrastructure (`crypto/pkcs7.h, linux/verification.h`). The two primary functions, `verify_signature_pkcs7_attached()` and `verify_signature_pkcs7_detached()`, handle the two primary signature formats.

Key Constants:

- `VERITY_FOOTER_SIGNED_LEN (196)`: Defines the precise byte length of the metadata region that is hashed for an attached signature. This fixed length is crucial for integrity.
- `VERIFYING_MODULE_SIGNATURE`: A constant flag passed to `pkcs7_verify()` instructing the kernel to check the signing certificate against the trusted Module Signatures keyring.

2. Analysis of `verify_signature_pkcs7_attached()`

This function validates signatures where the PKCS#7 blob is physically embedded within the verity metadata footer.

2.1. Initial Checks and Data Hashing

The process begins by reading the blob size and performing a strict bounds check:

```
56      blob_sz = le32_to_cpu(meta->pkcs7_size);
57      if (!blob_sz || blob_sz > VERITY_PKCS7_MAX)
58          return -EINVAL;
```

If the size is invalid, the function immediately rejects the signature with -EINVAL, preventing resource allocation failure or security risks from overly large blobs.

Next, the local digest of the signed data is computed. This is the hash of the data that the signature *claims* to cover:

```
60      ret = compute_footer_digest(meta, digest);
61      if (ret)
62          return ret;
```

The external function `compute_footer_digest()` calculates the SHA-256 hash over the first `VERITY_FOOTER_SIGNED_LEN` bytes of the meta structure and places the result in the 32-byte digest array.

2.2. PKCS#7 Parsing and Authentication

The raw binary PKCS#7 blob is parsed into a structured kernel object:

```
64      pkcs7 = pkcs7_parse_message(meta->pkcs7_blob, blob_sz);
65      if (IS_ERR(pkcs7))
66          return PTR_ERR(pkcs7);
```

If the structure is malformed or invalid, `IS_ERR()` catches the kernel pointer error and returns it immediately.

```
68      ret = pkcs7_verify(pkcs7, VERIFYING_MODULE_SIGNATURE);
69      if (ret)
70          goto out_free;
```

`pkcs7_verify()` performs two vital checks:

1. **Cryptographic Validation:** It verifies the cryptographic integrity of the digital signature itself using the public key in the certificate.
2. **Keyring Trust:** It checks the certificate's authenticity against the trusted public keys loaded into the kernel (the `VERIFYING_MODULE_SIGNATURE` keyring). If the signing key is not trusted, this call returns an error, preventing untrusted metadata from being used.

2.3. Final Integrity Check (The Hash Match)

After the signature is authenticated, the function retrieves the **signed digest** embedded within the PKCS#7 message structure:

```
72      ret = pkcs7_get_digest(pkcs7, &signed_hash, &signed_hash_len, &signed_hash_algo);
73      if (ret)
74          goto out_free;
```

The final integrity verification is a triple check performed on the signed content:

```
76      if (signed_hash_algo != HASH_ALGO_SHA256 || signed_hash_len != 32 ||
77          memcmp(signed_hash, digest, 32) != 0)
78          ret = -EKEYREJECTED;
```

1. **Algorithm Check:** Ensures the signed hash is indeed SHA-256 (`HASH_ALGO_SHA256`).
2. **Length Check:** Confirms the digest is exactly 32 bytes long.
3. **Content Match:** `memcmp()` performs a byte-by-byte comparison between the locally computed hash (digest) and the signed hash (`signed_hash`). If they do not match, the signed data was tampered with, and the signature is rejected with the specific code `-EKEYREJECTED`.

3. Analysis of `verify_signature_pkcs7_detached()`

This function handles scenarios where the signature blob is separate from the metadata buffer (`meta_buf`). The verification workflow is similar but requires an additional step to explicitly link the data and the signature.

3.1. Hashing and Parsing

The local digest is calculated over the entire metadata buffer using the external helper:

```
112     ret = sha256_buf(meta_buf, meta_len, digest);
113     if (ret)
114         return ret;
```

The PKCS#7 blob is parsed from the detached buffer (`sig_buf`):

```
116     pkcs7 = pkcs7_parse_message(sig_buf, sig_len);
```

3.2. Supplying Detached Data

Before verification, the kernel's PKCS#7 parser must be told which data buffer the signature applies to:

```
124     ret = pkcs7_verify(pkcs7, VERIFYING_MODULE_SIGNATURE);
125     if (ret)
126         goto out_free;
```

This call links the parsed signature message (pkcs7) to the actual metadata (`meta_buf`). This is the key difference from the attached function, which relies on the signature being physically adjacent to its data.

3.3. Verification and Final Check

The authentication (`pkcs7_verify()`) and digest retrieval (`pkcs7_get_digest()`) steps are identical to the attached verification process. The same final triple check is performed to ensure the integrity of the data has not been compromised after it was signed:

```
132     if (signed_hash_algo != HASH_ALGO_SHA256 || signed_hash_len != 32 ||
133         memcmp(signed_hash, digest, 32) != 0)
134         ret = -EKEYREJECTED;
```

4. Resource Cleanup

Both verification functions use a common resource cleanup block via a `goto out_free` label. This ensures that the memory allocated for the structured PKCS#7 message (`struct pkcs7_message *pkcs7`) is always released using `pkcs7_free_message(pkcs7)` before the function exits, regardless of success or failure. This is critical for kernel stability and memory management.

End of Document